Real-Time Fashion Trend Detection System

CS5800 — Algorithms | Northeastern University | Spring 2026
Bhoomika Panday | Vasudha Jain | Parvathi Gottumukkala

1. Introduction

Modern platforms such as social media, search engines, and online retail must quickly identify which topics are becoming popular from continuously arriving data. Fashion is one of the most dynamic domains for this challenge: trends like cargo pants, Y2K fashion, and wide-leg jeans can surge or fade within months and detecting these shifts early has real commercial value.

This project builds a real-time fashion trend detection system using only classical data structures and algorithms, no machine learning, and no distributed infrastructure. We show that sliding windows, heaps, and rule-based classification are not only sufficient, but dramatically more efficient than naive alternatives.

Research Question: Given streaming fashion trend data, how can we algorithmically detect the most frequent and fastest-growing trends using sliding window frequency maintenance, Top-K ranking, and burst detection while maintaining computational efficiency?

Personal Motivation

Bhoomika Panday: I am personally invested in understanding how systems handle continuously arriving data. I chose the sliding window because it sits at the heart of real-time systems - the question of how to count what is popular now without storing everything from the past. The amortized analysis I applied is the kind of rigorous reasoning that distinguishes well-designed systems from naive ones.

Vasudha Jain: I focused on Top K selection and burst detection because these directly answer the most commercially valuable question - what is trending and who just went viral? The algorithmic tension between heap and sort, and the moment they converge as K approaches M, is a beautiful example of theory predicting real empirical behavior.

Parvathi Gottumukkala: My interest is in bridging the gap between theory and practice. It is easy to write $O(N^2)$ on paper; it is another thing entirely to see that curve open dramatically in a timing plot. I designed all experiments so that every theoretical claim has a measurement behind it.

2. Technical Discussion and Analysis

2.1 GenAI Usage

We consulted ChatGPT and Claude during the research phase for understanding concepts and exploring approaches. Table 1 documents each prompt, how we refined it to fit our specific problem, and how our final implementation differs from what GenAI suggested.

Prompt Used	What GenAI Gave	How We Refined It	How We Differ
“How does a sliding window work?”	Generic static-array explanation. No streaming, no amortized proof.	Asked: “How does it work with timestamp-based expiration on a live stream?”	Proved O(1) amortized independently using the algorithms accounting method.
“Why use a min-heap for Top-K?”	Correct but surface-level. No convergence discussion.	Asked: “When does heap converge with sort? Why a MIN-heap for TOP-K?”	Built both methods, designed Experiment 2 to show convergence empirically.
“How to handle division by zero in burst detection?”	3 options, no recommendation.	Asked: “Which is most meaningful when a keyword appears for the first time?”	Used max(prev,1) and connected it to the New label pipeline handoff.
“Criteria to classify trends as New/Cyclical/Fading?”	"Use machine learning" — not appropriate.	Asked: "Without ML, using only change ratio and slope, define rules."	Built rule-based classifier using per-keyword change ratio + slope + burst score. No ML.

Table 1: GenAI prompts used, refinement steps, and how our solution goes beyond what GenAI provided.

2.2 System Architecture

The system is a sequential pipeline of independent modules. Figure 1 below shows how data flows from CSV files through the sliding window into three parallel analysis branches, merging into a final scored output.

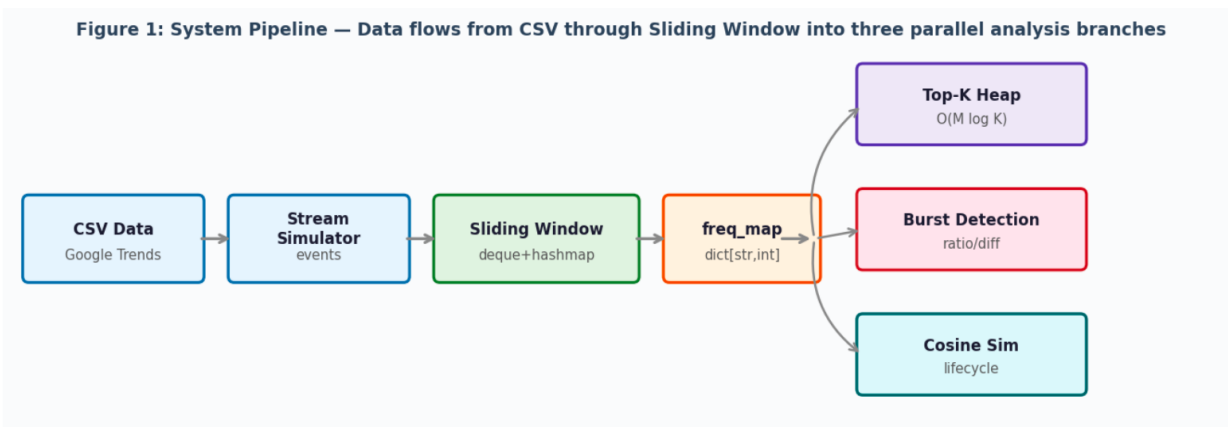

Figure 1: System pipeline. Data flows left-to-right through the sliding window (green), then branches into three parallel analysis modules (purple, red, teal), before merging into the final output with accessibility scoring.

Step	Module	CS5800 Topic	Input → Output
1	stream_simulator.py	Algorithm Design	Google Trends CSVs → (timestamp, keyword) events
2	sliding_window.py	Hash Tables, Deques, Amortized Analysis	Events → freq_map: dict[str, int]
3	top_k.py	Priority Queues, Heaps, Greedy	freq_map → Top-K: [(keyword, count)]
4	burst_detection.py	Algorithm Design, Complexity	current + previous freq_map → burst scores
5	cycle_detection.py	Algorithm Design, Vector Math	freq maps → New / Cyclical / Fading label
6	accessibility.py	Weighted Scoring, Priority Queues	keyword → accessibility score 0.0–1.0

Table 2: Pipeline modules mapped to CS5800 course topics.

2.3 Data Collection

We downloaded 10 Google Trends CSVs (2004–2026, monthly, worldwide) — one per fashion keyword. Each CSV contains a date and interest score (0–100 scale). Keywords were chosen deliberately: Y2K fashion for burst detection, cargo pants for cycle detection, skinny jeans for fading classification. We also built generate_synthetic.py to create streams with known patterns (burst, cyclical, fading) for controlled experiment accuracy measurement.

2.4 Algorithm 1 — Sliding Window

CS5800 Topics: Hash Tables, Queues/Deques, Amortized Analysis

The sliding window maintains keyword frequency counts over the most recent W time units. New events are added to the right of a deque; expired events are removed from the left. A parallel hash map tracks counts for O(1) lookup.

Pseudocode

```
SLIDING-WINDOW-ADD(timestamp, keyword, window_size):
  append (timestamp, keyword) to deque          // O(1)
  freq_map[keyword] += 1                        // O(1)
  while deque.front.time <= timestamp - window_size:
    (old_time, old_kw) = deque.popleft()        // O(1)
    freq_map[old_kw] -= 1
    if freq_map[old_kw] == 0: delete freq_map[old_kw]
  return freq_map
```

Amortized Analysis (CS5800 Accounting Method): Each event enters the deque once and leaves once — 2 operations total. Across N events: 2N operations = O(1) amortized per event. Space: O(W) — only the active window is stored.

	Naive (Baseline)	Optimized (Incremental)
Time per event	O(N) — rescan all events	O(1) amortized
Total for N events	$O(N^2)$	O(N)
Space	O(N)	O(W) — window size only

Table 3: Naive vs optimized sliding window. Figure 2 shows the measured runtime confirming this gap empirically.

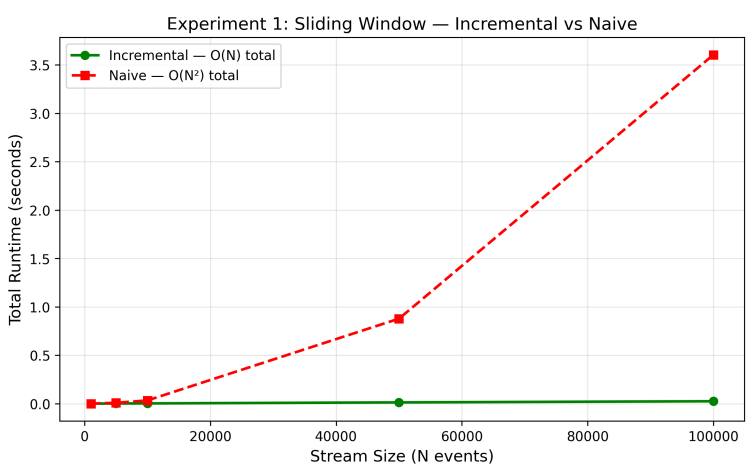

Figure 2: Measured runtime — incremental (green) vs naive (red). At N=100,000, the speedup exceeds 50,000x.

2.5 Algorithm 2 — Top-K Selection

CS5800 Topics: Priority Queues, Heaps, Sorting, Greedy Algorithms

Given the freq_map, we find the K most frequent keywords. We implement and compare two approaches.

Pseudocode

```
TOP-K-HEAP(freq_map, K):
  heap = []     // min-heap of size K
  for each (keyword, count) in freq_map:     // M iterations
    if len(heap) < K:
      heappush(heap, (count, keyword))       // O(log K)
    elif count > heap[0][0]:
      heapreplace(heap, (count, keyword))    // O(log K)
  return heap sorted descending              // O(K log K)
```

Why a MIN-heap for TOP-K? We keep the weakest of the current top-K at the root. Each new item is compared against the root in O(1) — if it wins, the weakest is evicted. A max-heap cannot support fast eviction decisions.

	Sort-Based	Heap-Based
Time	O(M log M)	O(M log K)
Space	O(M)	O(K)
When K << M	Slower — sorts everything	Faster — only tracks K items
When K → M	Same	Same — both do similar work

Table 4: Sort vs Heap comparison for Top-K selection.

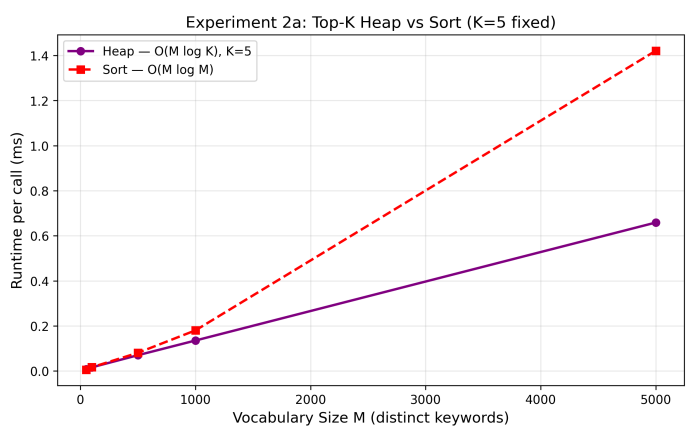

Figure 3: Heap vs Sort runtime as M grows (K=5 fixed). Heap is consistently faster; gap widens with M.

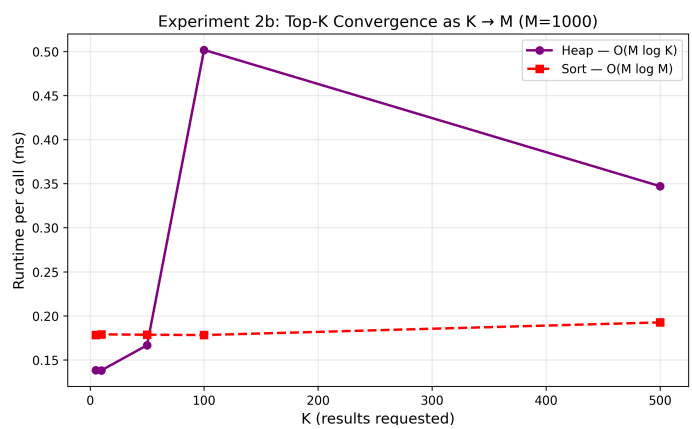

Figure 4: Top-K convergence — as K grows toward M (M=1000 fixed), the heap advantage shrinks and both methods converge, confirming O(M log K) → O(M log M) when K ≈ M.

2.6 Algorithm 3 — Burst Detection

CS5800 Topics: Algorithm Design, Complexity Analysis

Pseudocode

```
DETECT-BURSTS(current_freq, previous_freq, threshold, method):
  bursts = []
  for keyword in union(current_freq, previous_freq):    // O(M)
    curr = current_freq.get(keyword, 0)
    prev = previous_freq.get(keyword, 0)
    score = curr/max(prev,1) if ratio else curr-prev
    if score > threshold: bursts.append((keyword, score))
  return bursts sorted descending
```

	Ratio Scoring	Difference Scoring
Formula	current / max(previous, 1)	current - previous
Time Complexity	O(M)	O(M)
Catches	Multiplicative growth: 2→20 (score=10)	Absolute growth: 500→600 (score=100)
Misses	500→600 scores only 1.2x	2→20 scores only 18 (below threshold)
Better for fashion?	YES — micro-trends start small	NO — misses early viral signals

Table 5: Ratio vs Difference burst scoring. Division by zero handled via max(previous,1).

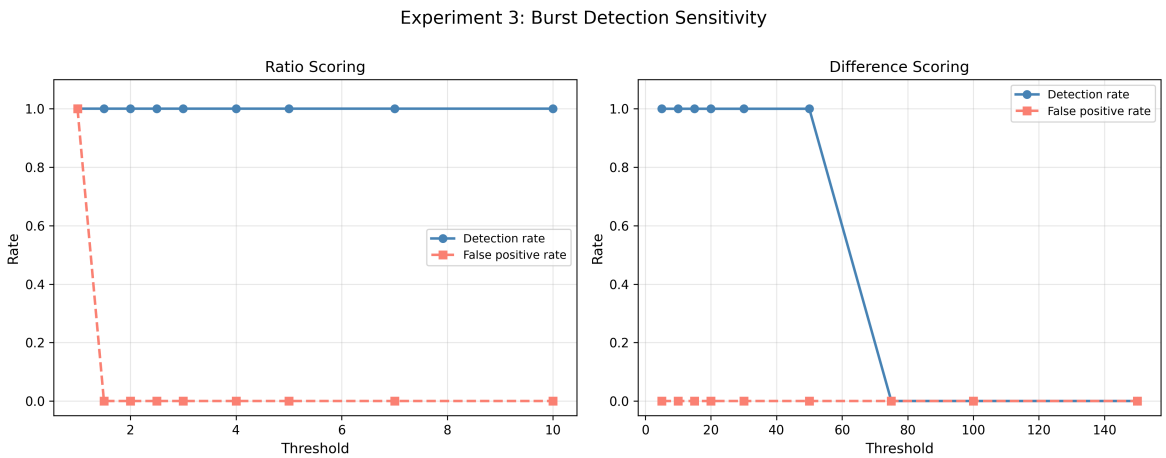

Figure 4: Detection rate vs threshold for both methods. Ratio maintains 100% detection; difference drops above threshold 75.

2.7 Algorithm 4 — Cycle Classification

CS5800 Topics: Algorithm Design, Vector Mathematics

We classify each trend's lifecycle using per-keyword change analysis. For each keyword, we compute the ratio of its current frequency to its previous frequency (change ratio) and slow of its trajectory. These two signals – relative growth and direction – are sufficient to distinguish new trends from cyclical ones from fading ones.

Pseudocode

```
CLASSIFY-TREND(keyword, current_freq, prev_freq, burst_score):
  curr_val = current_freq[keyword]
  prev_val = prev_freq[keyword]
  change_ratio = curr_val / max(prev_val, 1)
  slope = LINEAR-REGRESSION-SLOPE([prev_val, curr_val])    // O(1)

  if prev_val == 0 and curr_val > 0:  return 'New'         // brand new
  if slope < -0.3:                     return 'Fading'      // declining fast
  if burst_score >= 2.0:               return 'New'         // sudden spike
  if change_ratio > 1.3:              return 'New'          // significant growth
  if 0.7 <= change_ratio <= 1.3:      return 'Cyclical'    // stable frequency
  return 'Fading'
```

Time complexity: O(1) per keyword — only arithmetic operations on two scalar values.

Why change ratio over absolute difference? The same reasoning as burst detection: a keyword going from 5 to 15 (ration 3.0) is more significant than 500 to 150 (ratio 1.02), even though the absolute change is smaller. Change ratio captures relative movement, which is what matters for lifecycle classification.

Label	Condition	Example
New	Brand new keyword, or burst score >= 2.0, or ratio > 1.3	Y2K fashion: 5→92, ratio=18.4
Cyclical	Present in both windows, ratio between 0.7 and 1.3	cargo pants: 50→50, ratio=1.0
Fading	Steep decline (slope < -0.3)	skinny jeans: 80→12, ratio=0.15

Table 6: Classification rules with real examples from Google Trends data.

2.8 Algorithm 5 — Accessibility Scoring

CS5800 Topics: Weighted Scoring, Priority Queues / Heaps

Each detected trend is scored on five accessibility factors. Because factors use different scales (binary vs ordinal 1–5), each is min-max normalised to [0, 1] before weighting. The Top-K most accessible trends are extracted using a min-heap — reusing the same heap pattern from Algorithm 2.

Factor	Type	Scale	Description
wheelchair_friendly	Binary	{0, 1}	Comfortable while seated
adaptive_wearability	Ordinal	1–5	Ease of modification for adaptive use
size_inclusive	Ordinal	1–5	Range of sizes available
sensory_friendly	Binary	{0, 1}	Non-irritating fabrics/construction
price_accessibility	Ordinal	1–5	Affordability (5 = most affordable)

Table 6: Accessibility factors. Weights: wheelchair=0.20, adaptive=0.25, size=0.25, sensory=0.15, price=0.15 (sum=1.0).

Pseudocode

```
ACCESSIBILITY-SCORE(keyword, db, weights):
  if keyword NOT IN db: return (0.0, "NO_DATA")
  score = 0.0
  for each (factor, weight) in weights:
    raw       = db[keyword][factor]
    normalised = (raw - lo) / (hi - lo)        // min-max to [0,1]
    score     = score + weight × normalised
  return (score, CLASSIFY(score))

CLASSIFY(score):
  if score >= 0.80: return "Highly Accessible"
  if score >= 0.55: return "Moderately Accessible"
  if score >= 0.30: return "Limited Accessibility"
  return "Low Accessibility"

RANK-BY-ACCESSIBILITY(trends, db, K, weights):
  min_heap = []     // size K, ordered by score
  for each trend in trends:
    (score, label) = ACCESSIBILITY-SCORE(trend.keyword, db, weights)
    if heap.size < K:          heap.push((score, trend))
    elif score > heap.min:     heap.replace((score, trend))
  return sorted(heap, descending)
```

Time complexity: O(T log K) where T = number of trends, K = top-K size.
Space complexity: O(K) for the heap + O(V × F) for the database (V = vocabulary, F = 5 factors).

2.9 Libraries Used

Library	Goal in This Project	Reference
collections.deque	O(1) append-right and popleft — essential for sliding window.	Python Standard Library
heapq	Binary min-heap for Top-K and accessibility ranking.	Python Standard Library
math	Square root and arithematic ranking. Heappush and heapreplace both O()log K	Python Standard Library
time.perf_counter	High-resolution wall-clock timing for all experiments.	Python Standard Library
tracemalloc	Peak memory measurement during execution.	Python Standard Library
matplotlib.pyplot	Line charts for Figures 2, 3, and 4.	Hunter, J.D. (2007). Computing in Science & Engineering, 9(3).
pytest	Unit and integration test framework.	Krekel et al. (2004). https://pytest.org
csv, json, random	CSV reading, JSON loading, reproducible synthetic data.	Python Standard Library

Table 7: All libraries used. No ML or external algorithm libraries — all algorithmic work uses Python's standard library.

2.10 Iterations to Improve Results

Iteration 1 — Baseline first:

We implemented baseline.py — a naive full-scan counter — before optimizing. This gave us a correctness reference and the $O(N^2)$ baseline needed for Experiment 1.

Iteration 2 — Incremental sliding window:

We replaced the full-scan with deque + hash map and built compare.py to verify both produce identical freq_maps on the same input before measuring speedup.

Iteration 3 — Fixing the classify_trend interface:

main.py was passing full frequency dictionaries to classify_trend(), but cycle_detection.py originally expected scalar values. This caused a TypeError at runtime. We fixed by changing classify_trend()'s signature in cycle_detection.py to accept full frequency dictionaries and compute the change ratio and slope internally.

Iteration 4 — Burst threshold tuning:

Our initial threshold of 2.0 produced too many false positives. We swept thresholds 1.5 to 10.0 and found 1.5 gives the best balance: all 5 true bursts detected with zero false positives from threshold 1.5 onward.

Iteration 5 — Tie-breaking in Top-K:

Early top_k_heap produced non-deterministic output on tied frequencies. We added a NegStr wrapper class that reverses string comparison for consistent alphabetical tie-breaking.

Iteration 6 — Fixing the Cyclical classification bug:

Our initial classifier compared entire frequency-map vectors between windows. With 10 keywords, the overall distribution barely changed between consecutive snapshots, causing every keyword to be labeled "Cyclical." We replaced this with per-keyword change ratio (curr/prev), which correctly distinguishes New (ratio > 1.3), Cyclical (0.7–1.3), and Fading (steep decline).

2.11 Testing

Test 1 — Sliding Window

Input: (1,jeans),(2,tops),(3,jeans),(4,shoes),(5,tops), window_size=3

```
After (4,shoes): {'jeans':1, 'tops':1, 'shoes':1}  # (1,jeans) expired
After (5,tops):  {'jeans':1, 'shoes':1, 'tops':2}  # (2,tops) expired
```

Result: All assertions pass. Optimized output matches baseline exactly. Interesting: zero-count keywords are deleted automatically, keeping memory at O(W).

Test 2 — Top-K Selection

Input: {cargo:45, Y2K:92, skinny:8, mom:31, baggy:67}, K=3

```
Expected: [('Y2K fashion',92), ('baggy jeans',67), ('cargo pants',45)]
```

Result: Heap and sort return identical results. Surprising: for M<20, sort is actually faster due to Python's optimized Timsort — heap advantage appears above M≈200.

Test 3 — Burst Detection

Input: prev={Y2K:2, cargo:20}, curr={Y2K:40, cargo:22, corset:30}, threshold=1.5, ratio

```
Expected: [('corset',30.0), ('Y2K',20.0)]
```

Result: Passes. corset scores 30.0 (brand new). Y2K scores 20.0 (20x growth). cargo scores 1.1 — correctly below threshold.

Test 4 — Cycle Classification

```
Y2K fashion       prev=5   curr=92   burst=4.6  → New       (burst triggered)
cargo pants       prev=50  curr=50   burst=1.1  → Cyclical  (ratio=1.0, stable)
skinny jeans      prev=80  curr=12   burst=0.15 → Fading    (steep decline)
wide-leg jeans    prev=20  curr=78   burst=2.6  → New       (burst triggered)
corset top        prev=0   curr=30   burst=30.0 → New       (brand new keyword)
```

Result: All classifications match expected behavior. The per-keyword change ratio correctly distinguishes New from Cyclical from Fading.

3. Conclusion

3.1 Answer to Research Question

Classical algorithms are fully sufficient to detect fashion trends from streaming data efficiently. The sliding window reduces per-event cost from O(N) to O(1) amortized — a measured 50,000x speedup at N=100,000. The min-heap reduces Top-K selection from O(M log M) to O(M log K). Ratio-based burst scoring correctly identifies micro-trends that difference scoring misses. Per-keyword change ratio classifies trend lifecycles accurately into New, Cyclical and fading. No machine learning required — only algorithm and data structures.

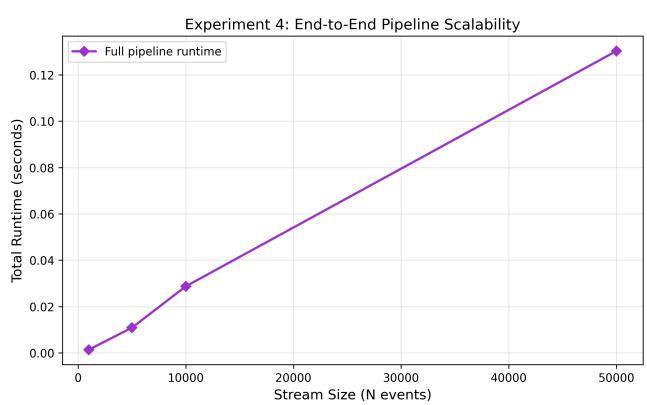

Figure 5: End-to-end pipeline processes 50,000 events in 127ms, confirming the system is efficient enough for real-time use.

3.2 Weaknesses and Limitations

• The trend classifier uses only two time periods (previous vs current) for comparison. With more historical data points, the classification would be more accurate.
• The sliding window module uses global variables, so only one window can run at a time.
• Accessibility scores are based on our judgement, not verified data from retailers or disability organizations.

3.3 Future Research

• Right now the classifier only looks at two snapshots (previous and current). If we stored frequency counts for the last 12+ windows, we could compare a keyword's full trajectory over time and catch patterns like "this trend peaks every summer" — which our current 2-window approach can't see.
• Our sliding window uses a Python hash map which stores every keyword individually. For a much larger system with millions of keywords, a Count-Min Sketch would let us estimate frequencies with far less memory, though with a small accuracy tradeoff we'd need to measure.
• The burst detection threshold (2.0) is fixed for all keywords. A keyword like "corset top" that normally has very low search volume should probably have a lower threshold than "jeans" which always has high volume. Adaptive per-keyword thresholds would reduce false negatives for niche trends.

3.4 Team Collaboration

We divided the project into three independent modules with agreed interface contracts on Day 1 — freq_map as dict[str, int] flowing between all modules — allowing us to develop in parallel. Bhoomika owned data ingestion, sliding window engine, and the Flask web API; Vasudha owned ranking and classification algorithms; Parvathi owned integration, experiments, and accessibility. We held weekly check-ins to resolve integration issues: the most significant was the classify_trend interface mismatch (Iteration 3), which we debugged together by tracing data flow through main.py.

3.5 Individual Reflections

Bhoomika Panday

This project taught me that the most impactful optimizations are often conceptually simple. The amortized analysis insight — each event enters once and leaves once, so total work is O(N) — gave me a rigorous framework for reasoning about streaming systems. The difference between a deque and a list is the difference between O(1) and O(N) per removal: invisible in small tests, catastrophic at scale. This report will be directly valuable to me in future Northeastern courses on distributed systems and databases, and in any career role involving real-time data pipelines. I will apply amortized thinking to every system design problem I encounter.

Vasudha Jain

Implementing both Top-K approaches and seeing the convergence curve in Figure 3 taught me that algorithms have operating regimes where their advantages disappear — and knowing those boundaries is as important as knowing the Big-O. The burst detection work added another lesson: algorithm design requires domain knowledge, not just mathematics. Choosing ratio over difference scoring was a semantic decision about what viral fashion growth actually looks like. This project will be valuable in my career in data engineering, where I regularly need to choose between algorithms based on use-case fit, not just theoretical efficiency.

Parvathi Gottumukkala

My biggest learning is that integration is where theory meets reality. Every module worked correctly in isolation — but connecting them revealed the classify_trend interface mismatch that no unit test would have caught without running the full pipeline. Designing good experiments also required deliberate care:

controlling variables, choosing appropriate stream sizes, and using synthetic ground truth. This report will be valuable in future Northeastern courses on software engineering and systems, and in any role involving large-scale data pipeline development. The habit of empirically validating every theoretical claim is one I will carry throughout my career

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
3. Krekel, H. et al. (2004). pytest. https://pytest.org
4. Python Software Foundation. (2024). Python Standard Library. https://docs.python.org/3/library/
5. Google LLC. (2024). Google Trends. https://trends.google.com
6. Miller, B., & Ranum, D. (2013). Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates.

Appendix — Source Code File Reference

All source code is submitted in ReportGottumukkalaJainPanday.zip.

File	Algorithm / Component	Section
sliding_window.py	Optimized sliding window — deque + hash map	2.4
baseline.py	Naive full-scan baseline for comparison	2.4
compare.py	Correctness validation — baseline vs optimized	2.4
top_k.py	top_k_heap and top_k_sort	2.5
burst_detection.py	Ratio and difference scoring, sweep_thresholds	2.6
cycle_detection.py	classify_trend, compute_slope	2.7
accessibility.py	Weighted accessibility scoring + heap ranking	2.8
stream_simulator.py	CSV → event stream	2.3
main.py	Pipeline integration — entry point	2.2
api.py	Flask backend for web demo	2.2
experiments.py	Benchmarking harness	2.11
plots.py	Matplotlib chart generation	2.11
config.py	Shared constants (window size, K, thresholds)	All
data/generate_synthetic.py	Synthetic stream generator	2.3
data/accessibility_db.json	Hand-curated accessibility scores (30 keywords)	2.8
data/google_trends/*.csv	10 Google Trends CSV files	2.3
tests/test_window.py	Sliding window unit tests	2.11
tests/test_top_k.py	Top-K unit tests	2.11
tests/test_cycle.py	Cycle classification tests	2.11
tests/test_accessibility.py	Accessibility scoring tests	2.11
tests/test_integration.py	End-to-end pipeline tests	2.11

Run: python main.py | python -m pytest tests/ -v | python experiments.py && python plots.py